BETO Framework: An Epistemic Governance Protocol for LLM-Assisted Software Specification

Alberto Ramírez Independent Researcher and Software Architect Version 1.0 — March 2026

Abstract

Large Language Models (LLMs) excel at generating syntactically correct, structurally coherent outputs from incomplete specifications. This completion capability, central to their utility, becomes a liability in software engineering: models silently invent fields, assume architectures, and collapse ambiguities without registering them — producing systems that function for average cases but cannot be audited, traced, or governed. We introduce **BETO**, an epistemic governance protocol that formalizes the boundary between what has been declared by the operator and what has been assumed by the model. BETO enforces three epistemic states — `DECLARED`, `NOT_STATED`, and `INFERRED` — across an 11-step specification and materialization pipeline. It introduces formal mechanisms for detecting and blocking undeclared operations (`BETO_GAP`), for tracing every line of generated code to an authorized specification decision (`TRACE_REGISTRY`), and for preserving human authority over system topology through operator-controlled gates. We report on three systems fully specified under BETO — including BETO_EXECUTOR, the automated pipeline that implements BETO itself — demonstrating that disciplined epistemic governance produces materializeable specifications with complete traceability from requirement to code, without restricting the productive capabilities of LLMs.

---

1. Introduction

The adoption of LLMs in software development has created a new class of engineering problem. When a developer asks a model to design or implement a system, the model does not acknowledge what it does not know — it completes. It invents field names because they seem reasonable. It assumes database schemas because they are conventional. It generates code that handles the common case but was never authorized by the operator.

This behavior is not a defect. Completion is the defining function of autoregressive language models. The defect is architectural: modern software development workflows have no formal mechanism to distinguish between what an operator declared and what a model inferred. The result is **epistemic debt** — a growing gap between the system that was specified and the system that was built.

Existing approaches address adjacent problems. Architecture Decision Records (ADRs) [1] document decisions after they are made, but do not govern the generation process. OpenAPI specifications [2] define contracts for external interfaces but do not constrain

internal design. Prompt engineering practices [3] guide model behavior through natural language but provide no formal enforcement. Test-Driven Development [4] validates behavior post-implementation but cannot detect silent scope expansion during specification.

BETO takes a different approach: **governance at the point of generation**. Rather than auditing what was built, BETO enforces what can be built — at each step of the specification pipeline — by requiring that every system element carry one of three formally defined epistemic states before any materialization is authorized.

The framework has been developed and iteratively refined over four years of application to real software systems. This paper describes BETO v4.2, the current stable version, and reports on its application to three systems of increasing complexity.

---

2. Background and Related Work

2.1 The Silent Completion Problem

When LLMs generate specifications or code from incomplete inputs, they engage in what we term **silent completion**: filling underdetermined regions of the output space with plausible content, without flagging that this content was not authorized. This phenomenon has been documented across code generation [5], requirements engineering [6], and system design tasks [7].

The consequences are systematic. In a study of LLM-assisted specification workflows, Ramírez (2022–2026) documented five recurring failure modes:

1. **Specification inflation**: the model generates capabilities the operator did not request.
2. **Fragile contracts**: data fields between components have no traceable origin.
3. **Epistemic debt**: the team cannot distinguish designed decisions from model completions.
4. **Audit failure**: there is no mechanism to trace code back to a specific specification decision.
5. **Silent scope expansion**: the model introduces dependencies and relationships not present in the original intent.

These failure modes share a root cause: the absence of a formal epistemic layer between the operator's intent and the model's output.

2.2 Existing Governance Mechanisms

Software engineering has developed several governance mechanisms that address related problems without solving the silent completion problem:

- **Model-Driven Engineering (MDE)** [8] provides formal model-to-code transformations but operates on complete, pre-specified models — it does not govern the process of creating those models with LLM assistance.

- **Behavior-Driven Development (BDD)** [9] formalizes acceptance criteria but does not constrain the design space between feature description and implementation.
- **Lightweight Architecture Documentation** [10] encourages decision logging but provides no enforcement mechanism for LLM-assisted generation.
- **Constitutional AI** [11] establishes behavioral constraints for LLMs at the model level but cannot enforce domain-specific epistemic rules at the specification level.

BETO fills this gap by operating as a **meta-protocol**: a governance layer above any specific LLM that enforces epistemic discipline throughout the specification and materialization process.

---

3. The BETO Framework

3.1 Design Principles

BETO is built on four core principles:

1. **Epistemic sovereignty of the operator.** Only the operator can authorize system elements. The model may propose; it cannot decide.
2. **Formalized ignorance.** What is not known must be registered, not resolved silently. Unknown elements become Open Questions (OQs) that block execution until the operator resolves them.
3. **Bounded inference.** Model inference is authorized exclusively in a controlled frontier (the initial elicitation phase, Steps 0–1). Beyond that frontier, inference is prohibited.
4. **Traceable materialization.** Every line of generated code must be traceable to an authorized specification element via a registered identifier.

The central definition of the framework follows directly from these principles:

> **BETO formalizes the ignorance of an AI.**

3.2 Epistemic States

Every element of a system under BETO specification carries exactly one of three epistemic states:

State	Definition	Effect on Execution
`DECLARED`	Explicitly defined by the operator	Enables execution
`NOT_STATED`	Not declared; cannot be inferred	Blocks execution — registered as Open Question
`INFERRED`	Derived from context by the executor	Authorized only in Steps 0–1. Prohibited after BETO_CORE_DRAFT closure

The `INFERRED` state is transitional: it must be resolved to `DECLARED` (by operator confirmation) or to `NOT_STATED` (by operator rejection) before the specification can proceed beyond Step 1.

This three-state model is the formal mechanism that prevents silent completion. A model operating under BETO cannot transition an element from `NOT_STATED` to implemented without explicit operator authorization.

3.3 Structural Node Taxonomy

BETO organizes system components into three formal node types:

ROOT (BETO raíz) The single structural trunk of the system. Generated directly from the operator's IDEA_RAW. Defines the System Intent, boundaries, invariants, and the conceptual capability map. There is exactly one ROOT per system.

PARALLEL (BETO_PARALELO) A functional node born from semantic independence. A component qualifies as PARALLEL if it can be fully specified using only external contracts: inputs, outputs, interfaces, and declared responsibilities. It does not require knowledge of the internal structure of other components. Parallel nodes can be developed independently.

SUBBETO A structural refinement node born from ambiguity. A component qualifies as SUBBETO when its design requires knowledge of the internal structure or algorithms of its parent. SubBETOs represent vertical decomposition to resolve structural ambiguity — not functional separation.

This distinction is operationally enforced through the **semantic independence test**:

> *Can this component be developed by an independent team given only a document describing its purpose, inputs, outputs, and contracts — without explaining the internal implementation of other components?*
>
> — YES → PARALLEL — NO → SUBBETO

3.4 Topological Governance: BETO_SYSTEM_GRAPH

The topology of a BETO-governed system is formalized in the `BETO_SYSTEM_GRAPH`, a mandatory artifact generated after structural classification and before any child nodes are expanded. The graph uses three authorized edge types:

Edge Type	Meaning
`FUNCTIONAL_BRANCH`	ROOT → PARALLEL: born from functional autonomy
`STRUCTURAL_REFINEMENT`	Any node → SUBBETO: born from structural ambiguity
`DECLARED_DEPENDENCY`	Node A → Node B: A uses B's output contract as declared input

The BETO_SYSTEM_GRAPH must pass nine topological validations — including single-root verification, acyclicity, and complete node provenance — before achieving `VALIDATED` status. An un-validated graph blocks all downstream steps. Once validated, the graph

cannot be modified retroactively: it is the sole authority over the system's topology for the remainder of the specification cycle.

3.5 The BETO_GAP Mechanism

A BETO_GAP is a formal detection event triggered when the executor encounters an operation that would require unauthorized inference — an element that is NOT_STATED but would need to be implemented. The protocol mandates two resolution paths:

Path A — Derivable from System Intent:

```
BETO_GAP [RESOLVED: BETO_ASSISTED]
Justification: [explicit reasoning from declared System Intent]
→ Executor continues. Gap remains traceable in log.
```

Path B — Not derivable from System Intent:

```
BETO_GAP [ESCALATED: requires operator]
→ Mandatory halt. Executor cannot continue without explicit operator
declaration.
→ No silent resolution exists.
```

The BETO_GAP mechanism is the enforcement layer for the principle of formalized ignorance. It converts every encounter with an undeclared element into a traceable event — either a justified derivation or a formal request for operator input.

3.6 Traceability: TRACE_REGISTRY and BETO-TRACE Annotations

Every BETO_CORE specification generates a TRACE_REGISTRY: a catalogue of authorized traceability identifiers following the pattern:

```
SISTEMA.SEC<N>.<TYPE>.<ELEMENT>
```

Where SISTEMA is the node name in uppercase, SEC<N> is the specification section number, TYPE is one of eleven authorized element types (INTENT, SCOPE, INPUT, OUTPUT, UNIT, FIELD, TRACE_FIELD, CONCEPT, PHASE, DECISION, RISK, CONSTRAINT), and ELEMENT is a descriptive identifier.

Example authorized identifiers from the BETO_EXECUTOR system:

```
BETO_MOTOR_RAZ.SEC4.UNIT.PASO_EJECUCION
BETO_MOTOR_RAZ.SEC8.DECISION.LLM_API_OPENAI_COMPATIBLE
BETO_GESTOR.SEC7.PHASE.PHASE_3_LECTURA_REANUDACION
BETO_MOTOR_COD.SEC5.INVARIANT.TRACE_VERIFIED_REQUIRED
```

Every file generated in Step 10 (Materialization) must annotate its structural elements with BETO-TRACE identifiers drawn exclusively from the TRACE_REGISTRY of the corresponding BETO_CORE. An identifier not present in the TRACE_REGISTRY is unauthorized: its presence constitutes a BETO_GAP [ESCALATED] and blocks delivery of that file.

This mechanism establishes a formal chain of custody from code element to specification section to operator-authorized decision.

---

4. The 11-Step Process

The BETO specification cycle consists of eleven steps divided into two closure phases:

Construction cycle (Steps 0–10):

Step	Artifact	Purpose
0	PASO_0_EVALUACION.md	Semantic eligibility assessment of IDEA_RAW
1	BETO_CORE_DRAFT.md	Root BETO_CORE generation — bounded inference frontier
2	BETO_CORE_INTERVIEW_COMPLETED.md	12-section structural interview to resolve OQs
3	STRUCTURAL_CLASSIFICATION_REGISTRY.md	Formal classification of all components
4	BETO_SYSTEM_GRAPH.md	Topology freeze and validation (9 checks)
5	BETO_CORE (children)	BETO_CORE generation for each authorized node
6	CIERRE_ASISTIDO.md	Assisted closure — all OQs resolved to SUCCESS_CLOSED
7	PHASE_*.md	Phase documents per BETO_CORE (8-section template)
8	MANIFEST + TRACE_REGISTRY	Individual manifests and authorized ID catalogues
9	MANIFEST_PROYECTO.md	Complete project manifest from validated graph
10	Source files	LLM-assisted code generation with BETO-TRACE annotations

Operational cycle (Step 11):

Step	Artifact	Purpose
11	FRAMEWORK_FEEDBACK.md + OPERATIONAL_LESSONS.md	Formal learning snapshot after first production operation

Human Gates. BETO defines three mandatory operator-controlled checkpoints:

- **G-1** (after Step 1): Operator approves or rejects the BETO_CORE_DRAFT before structural expansion.

- **G-2** (after Step 4): Operator approves or rejects the BETO_SYSTEM_GRAPH topology before child generation.
- **G-3** (after Step 9): Operator approves or rejects the complete specification before materialization begins.

Human gates are non-bypassable. An operator rejection does not trigger automatic regeneration — it triggers an operator-directed revision process. This prevents the framework from entering regeneration loops while preserving human authority over every major decision point.

4.1 Step 0 — Semantic Eligibility Assessment

IDEA_RAW must pass a formal eligibility check before the specification process begins. The assessment evaluates two simultaneous conditions:

1. **Creative intent**: does the IDEA_RAW express a will to create, transform, resolve, structure, or materialize something?
2. **Minimum semantic coherence**: does it contain enough conceptual core to identify at least one of: a recognizable object of creation, a functional direction, a transformation to perform, or a problem to solve — without arbitrary expansion?

The authorized outputs are three: `GO`, `GO_WITH_WARNINGS`, and `NO_GO`. Vagueness alone is not grounds for rejection; semantic vacuity is. A NO_GO triggers mandatory halt with no artifacts generated.

This step establishes the unique boundary between what BETO calls the *possible solution universe* and the *authorized solution universe*: only what passes the eligibility gate can become a system under BETO governance.

4.2 Step 10.5 — Post-Materialization Verification

For generator systems (systems whose purpose is to create, materialize, or validate products of other systems), BETO mandates a formal post-materialization verification step before any file can be declared `DELIVERED`:

- **L1 — Syntax verification**: deterministic static analysis of all generated `.py` files using `py_compile`. Any L1 failure constitutes a mandatory halt.
- **L2 — Import verification**: AST-based scan detecting imports referencing non-existent modules. L2 findings are reported to the operator for decision.
- **L3 — Semantic verification** (optional): a single LLM call over the complete file set, used only when L1+L2 pass and the operator requests semantic validation.

The **anti-loop rule** is absolute: the verifier is read-only. It never triggers automatic regeneration. If gaps are found, the operator decides whether to open a new materialization cycle — a new Step 10, not an extension of the current one.

---

5. BETO_EXECUTOR: Automated Implementation

BETO_EXECUTOR is the automated pipeline that implements the BETO Framework using two LLM backends: a reasoning motor for specification (Steps 0–9) and a code generation motor for materialization (Step 10).

5.1 Architecture

```
IDEA_RAW input
      |
      v
  [MOTOR RAZONAMIENTO: claude-sonnet-4-6]

  Step 0 --> Step 1 --> Step 2 --> Step 3 --> Step 4
              |                                |
             G-1                              G-2
            gate                             gate

  Step 5 --> Step 6 --> Step 7 --> Step 8 --> Step 9
                                               |
                                              G-3
                                             gate
      |
      v
  [MOTOR CODIGO: Qwen-Coder, local GPU]

  Plan derivation --> Scaffold generation -->
  LLM implementation --> TRACE_REGISTRY verification
      |
      v
  Verified source files (TRACE_VERIFIED status)
```

The system operates from a local infrastructure stack (LiteLLM gateway at `localhost:8000`, Qwen-Coder via vLLM at `localhost:8001` on an NVIDIA RTX 4090), enabling fully local materialization without external API dependency for code generation.

5.2 Structural Components

BETO_EXECUTOR was itself specified under BETO in a complete 11-node specification cycle. Its five parallel nodes represent the primary functional components:

Node	Type	Function
`BETO_EXECUTOR` (root)	ROOT	System intent, boundaries, invariants
`BETO_GESTOR_CICLO`	PARALLEL	Cycle state management and persistence
`BETO_MOTOR_RAZONAMIENTO`	PARALLEL	Step 0–9 LLM orchestration
`BETO_MOTOR_CODIGO`	PARALLEL	Step 10 code generation pipeline
`BETO_GATES_OPERADOR`	PARALLEL	Human gate interaction and decision capture

5.3 Key Engineering Decisions

Two-call generation for tail sections. For large documents (Steps 2 and 4), a single LLM call at `max_tokens=16384` consistently truncates the final sections — Section 12 (Consistency Pass) and Section 14 (Final Validation Status) — which appear at the end of the document. BETO_EXECUTOR implements a two-call strategy: Call 1 generates Sections 1–11 (or 1–13); Call 2 generates the tail section with the prior content as context. This guarantees structural completeness regardless of document size.

Template injection as context. Each step injects the corresponding BETO framework template before the IDEA_RAW in the LLM context. Without this, models generate plausible but non-BETO-compliant structure. With it, models produce artifacts that conform to the template schema, enabling deterministic validation.

Scaffold-then-implement code generation. Rather than generating code in a single pass, BETO_EXECUTOR builds a Python scaffold — with BETO-TRACE IDs from the TRACE_REGISTRY embedded in the module docstring — before invoking the code model. The code model receives the scaffold as a structural contract: it must preserve all BETO-TRACE annotations and implement the declared classes and function signatures. A post-generation verifier (`verificar_preservacion`) confirms that every scaffold ID is present in the final code, detecting deletions or corruptions introduced by the model.

BETO_STATE: Live Epistemic Context. From Step 2 onwards, each LLM call receives a `BETO_STATE.json` document as the first message in the conversation. This document captures the current epistemic state of the cycle: System Intent, declared boundaries, resolved Open Questions, active node topology, and gate decisions. Rather than requiring each call to re-derive system context from all prior artifacts, BETO_STATE provides a compact, structured summary that reduces context window consumption while maintaining epistemic coherence across steps.

5.4 Structural Validation at Gates

Before each human gate, BETO_EXECUTOR runs a deterministic, LLM-free structural validator against the artifacts generated in the preceding steps. Validation results are presented to the operator alongside the artifacts, enabling informed approval or rejection decisions. The validator is read-only and never triggers automatic regeneration: the operator's gate decision is always final.

Gate validation checks include: - **G-1 (7 checks)**: BETO_CORE_DRAFT section completeness, OQ formatting, System Intent presence, bounded frontier compliance. - **G-2 (16 checks)**: Interview Section 12 presence, BETO_SYSTEM_GRAPH validation status, topological constraints, DECLARED_DEPENDENCY consistency. - **G-3 (6 checks)**: MANIFEST_PROYECTO completeness, materialization file list presence, all BETO_COREs in SUCCESS_CLOSED state.

---

6. Empirical Evaluation

6.1 System 1: Dev Assistant

The first system specified under BETO Framework v4.2 was a software development assistant that analyzes local project structure and applies development tasks using a local LLM. The system operates fully offline.

Specification outcome:

Metric	Value
Total BETO nodes	6
PARALLEL nodes	2
SUBBETO nodes	3
Open Questions raised during interview	17
Open Questions resolved in assisted closure	17
Final state	SUCCESS_CLOSED (all nodes)
Phase documents generated	8
TRACE_REGISTRY identifiers authorized	74
Source files materialized	11
Files at TRACE_VERIFIED status	11

The system was delivered with zero silent completions. Every field in every data contract — including the `ComprehensionModel`, `StructuredTask`, and `ResolvedScope` entities — has a traceable origin in an operator-approved specification section.

6.2 System 2: BETO Artifact Evaluator

The BETO Artifact Evaluator is a quality assessment system for completed BETO cycles. It accepts a cycle's artifact directory and produces a structured evaluation report with per-dimension scores and one of three final recommendations: `CICLO VÁLIDO`, `CICLO CON DEUDA`, or `CICLO INVÁLIDO`.

The IDEA_RAW for this system was assessed as `GO_WITH_WARNINGS` at Step 0, with seven explicitly registered warnings covering undefined input contracts, unspecified scoring scales, and unresolved LLM integration conditions. All seven warnings were preserved as Open Questions through the specification pipeline and resolved during the assisted closure phase.

Specification outcome:

Metric	Value
Total BETO nodes	11
PARALLEL nodes	5
SUBBETO nodes	5

Initial Open Questions (GO_WITH_WARNINGS flags)	7
Total Open Questions during interview	12
Open Questions resolved	12
TRACE_REGISTRY identifiers authorized	138
Phase documents generated	15
Source files materialized	18
Files at TRACE_VERIFIED status	18

A notable outcome: the five SUBBETO nodes define the data contracts that govern cross-component communication. Because BETO required formal semantic independence testing (Step 3), the five PARALLEL nodes were developed with complete contractual clarity — no cross-component interface was discovered during implementation.

6.3 System 3: BETO_EXECUTOR (Self-Specification)

BETO_EXECUTOR was itself specified under BETO in a complete cycle. This self-specification served as the primary validation that the framework can govern a generator system — a system whose purpose is to execute the framework.

The specification applied the `REGLA_SISTEMAS_GENERADORES` rule, which mandates a `GENERATOR_RULES` artifact for any system that creates, materializes, or validates other systems. The resulting `GENERATOR_RULES_BETO_EXECUTOR.md` defines four minimum rules:

- **RULE_001**: Contract enforcement by construction — no critical output contract can depend on model compliance alone.
- **RULE_002**: Materialization retry scope — a localized failure does not authorize rerunning closed reasoning steps.
- **RULE_003**: Generator rules are mandatory — loaded and applied before every materialization cycle.
- **RULE_004**: Defensive output normalization — the generator normalizes model output before formal verification.

Specification outcome:

Metric	Value
Total BETO nodes	5 (root + 4 parallel)
Specification artifacts produced	38
TRACE_REGISTRY identifiers authorized	112
Source files materialized	14
Files at TRACE_VERIFIED status	14
BETO_GAP events during materialization	3 (all BETO_ASSISTED resolved)

The three BETO_GAP events during materialization were all derivable from the declared System Intent and resolved via the BETO_ASSISTED path — no operator intervention

required. This confirmed that the BETO_GAP mechanism successfully distinguishes derivable completions from undeclared assumptions.

---

7. Discussion

7.1 What BETO Adds to LLM-Assisted Development

BETO does not restrict LLM capabilities. It restricts which outputs of those capabilities can be accepted as system decisions. The distinction is architectural: BETO acknowledges that LLMs are powerful completion engines and channels that power within a formally bounded specification space.

The framework's primary contribution is the operationalization of **epistemic authority**: at every step of the development process, there is a clear, enforced answer to the question *"who authorized this?"* For a `DECLARED` element: the operator authorized it at a specific gate or during a specific interview response. For a `NOT_STATED` element: no one authorized it, and the system is blocked until someone does. For an `INFERRED` element within Steps 0–1: the framework authorized a bounded expansion — and the operator reviewed it at G-1.

7.2 The Traceability Chain

A fully BETO-governed system exhibits a complete chain of custody for every implementation element:

```
Source code line
    | (BETO-TRACE annotation)
    v
TRACE_REGISTRY identifier
    | (registry lookup)
    v
BETO_CORE section and authorized type
    | (gate approval record)
    v
Operator decision at human gate
    | (IDEA_RAW provenance)
    v
Original operator intent
```

This chain enables a form of software audit that is not available in conventional LLM-assisted development: given any line of generated code, one can trace it to the specific operator decision that authorized its existence.

7.3 Scalability: BETO_STATE

In systems with three or more parallel nodes, injecting all prior artifacts into each LLM call creates excessive context window consumption. BETO_STATE addresses this by maintaining a compact JSON document that captures the live epistemic state of the cycle.

Rather than re-reading all artifacts, each LLM call receives the current BETO_STATE as its first message, providing systemic context without full artifact injection.

BETO_STATE is updated by the executor after each completed step. It is an active epistemic record — not a summary — containing: resolved System Intent, declared boundaries, closed Open Questions, active topology, and gate decision history. In empirical testing, BETO_STATE reduced context window consumption by approximately 60–70% in multi-node specification cycles while maintaining full epistemic coherence.

7.4 Applicability Beyond BETO

The epistemic governance principles underlying BETO are not specific to the framework's particular templates or step sequence. They address a general problem in LLM-assisted engineering: the absence of formal mechanisms to distinguish operator intent from model completion. The core contributions — epistemic state tracking, the BETO_GAP detection and resolution protocol, TRACE_REGISTRY-based code traceability, and human gates at topology-critical decision points — are applicable to any workflow that combines LLM assistance with requirements for auditability and operator authority.

---

8. Conclusion

We have presented BETO, an epistemic governance protocol that formalizes the distinction between what is declared by the operator and what is completed by an LLM during software specification and materialization. The framework introduces three formal mechanisms — epistemic state tracking, BETO_GAP detection and resolution, and TRACE_REGISTRY-based traceability — that together enforce operator authority from initial idea to delivered source code.

Three systems have been fully specified and materialized under BETO: a software development assistant (6 nodes, 11 source files), a BETO artifact evaluator (11 nodes, 18 source files), and BETO_EXECUTOR itself (5 nodes, 14 source files). In all three cases, the framework produced materializeable specifications with complete traceability and zero silent completions — demonstrating that epistemic governance does not require sacrificing the productive capabilities of LLMs.

As LLM-assisted software development becomes standard practice, the engineering community will require formal mechanisms to audit, govern, and trace the outputs of model-assisted specification processes. BETO represents a concrete, field-tested contribution to that need.

The framework, its templates, and BETO_EXECUTOR are available as open artifacts. The author invites extension, critique, and application to domains beyond software engineering where the formalization of model-driven knowledge gaps is relevant.

References

[1] Nygard, M. (2011). *Documenting Architecture Decisions*. Cognitect Blog.

[2] OpenAPI Initiative. (2021). *OpenAPI Specification v3.1.0*. https://spec.openapis.org/

[3] White, J., et al. (2023). *A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT*. arXiv:2302.11382.

[4] Beck, K. (2002). *Test-Driven Development: By Example*. Addison-Wesley.

[5] Chen, M., et al. (2021). *Evaluating Large Language Models Trained on Code*. arXiv:2107.03374.

[6] Arora, C., et al. (2023). *Advancing Requirements Engineering through Generative AI: Assessing the Role of LLMs*. arXiv:2310.13976.

[7] Ahmad, A., et al. (2023). *Towards AI-Assisted Software Architecture: A Study on LLM-Based Design Decision Support*. arXiv:2310.18231.

[8] Schmidt, D.C. (2006). *Model-Driven Engineering*. IEEE Computer, 39(2), 25–31.

[9] Chelimsky, D., et al. (2010). *The RSpec Book: Behaviour Driven Development with RSpec, Cucumber, and Friends*. Pragmatic Bookshelf.

[10] Kruchten, P. (2004). *An Ontology of Architectural Design Decisions in Software Systems*. Groningen Workshop on Software Variability.

[11] Bai, Y., et al. (2022). *Constitutional AI: Harmlessness from AI Feedback*. arXiv:2212.08073.

Correspondence: Alberto Ramírez Framework documentation and implementation available at: BETO_EXECUTOR source repository Framework version: BETO v4.2 (2026-03-07)